

Lab 1: Flip array
Geavanceerde computerarchitectuur

(ULTRA-B-KUL-JLI1QS-2324)

Michiel Van Nieuwenhove

February 17, 2024



Group: MEIswe

Instructor: Alex Marinšek

1 Introduction

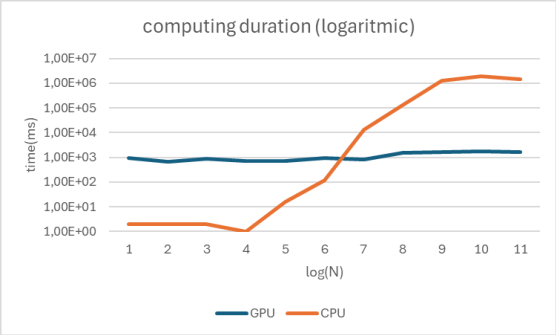
The assignment consists of inverting an array with the GPU, using CUDA. The advantage of this is to be able to execute a lot of threads in parallel.

2 Implementation

To implement this, a function 'flipArray()' is defined that takes as arguments an array and the size of this array. The function has the __global__ qualifier to define that it will run on the GPU and be called from the CPU. The index for every thread is set to:
threadIdx.x + blockIdx.x * blockDim.x
This ensures that each thread has a different index. After this each element in the array is assigned to the thread with the same index. This thread is responsible for placing that element on the inverted position in the output array. To call this function, first an input array is made, consisting of N elements. This is the array that is going to be inversed. The input-array is stored on the CPU. To work with this array on the GPU, it needs to be copied to the memory there. For this, the functions 'cudaMalloc()' and 'cudaMemcpy()' are used. The first one is used to allocate memory the size of the array, and the second one to copy the array from the memory of the CPU to that of the GPU. After these steps, the amount of threads and blocks can be decided. These are stored in dim3 datatypes. The block-size is set to 256 for example, and the grid-size to: (N + blockSize.x - 1) / blockSize.x
This calculation ensures that there are enough blocks to cover the entire array, even if the array size is not an exact multiple of the block size. Finally, the 'flipArray()'-function can be called with the predefined block-size and grid-size, and as arguments the array and its size. With 'cudaMemcpy()' the inverted array can be copied back to the CPU, and with 'cudaFree()' the allocated memory in the GPU can be freed.

3 Evaluation & Conclusion

In the figure below, the execution times of the algorithm are given for different amounts of elements in the array.



Evidently, executing the inversion algorithm on the CPU is very lightweight for small amounts of elements, but increases with bigger ones. By executing the algorithm on the GPU, it takes more time for the algorithm to assign and schedule each thread, but the execution time stays more or less constant over different array-sizes. This is due to the parallel execution of threads on the GPU.